

# AI-Assisted Software Development

## Mastering AI Coding Assistants

Mike Burton    4-6 November 2025

# Welcome to Mastering AI Coding Assistants

AI Coding assistants are revolutionising the way we write, debug, and understand code  
By leveraging advanced machine learning models, these tools:

- provide intelligent code suggestions
- automate repetitive tasks
- enhance overall productivity

In this module, we will:

- explore the fascinating world of AI-powered coding tools

# What is an AI Coding Assistant?

An AI Coding assistant:

- is a tool that leverages artificial intelligence to enhance the software development process
- provides various features to help developers write, debug, and understand code more efficiently

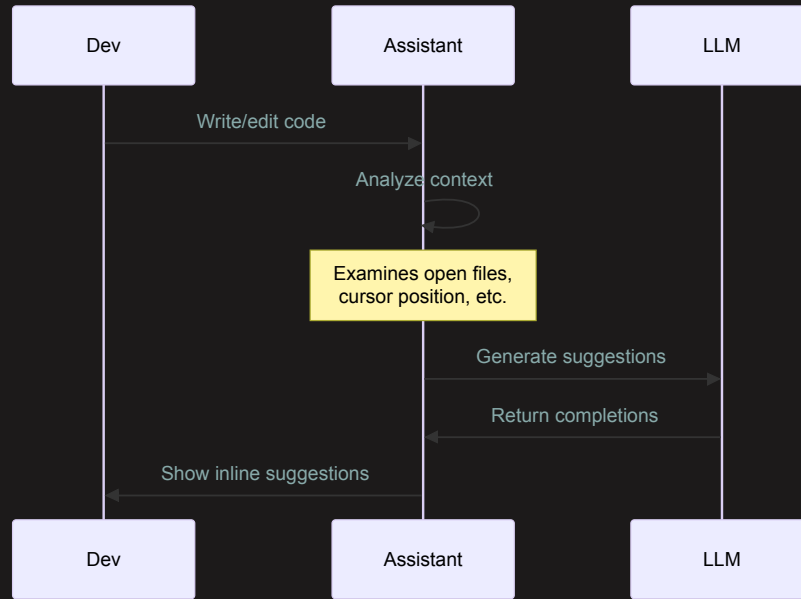
## Key Features:

- **Intelligent Code Completion:** Predicts and suggests the next lines of code based on context
- **Natural Language Code Generation:** Converts plain language instructions into code
- **Context-Aware Suggestions:** Offers relevant code snippets and solutions based on the current context.
- **Automated Documentation:** Generates documentation for your code automatically
- **Code Explanation:** Provides explanations for complex code segments

By integrating these features, AI Coding assistants aim to streamline the coding process, reduce errors, and boost overall productivity.

# How do AI Coding Assistants Work?

AI Coding assistants work by using a combination of techniques to understand the code and provide suggestions:



## Key Components:

- Large Language Models (LLMs)
- Context analysis
- IDE integration
- Real-time suggestion engine
- Version control integration

# Chat with Codebase

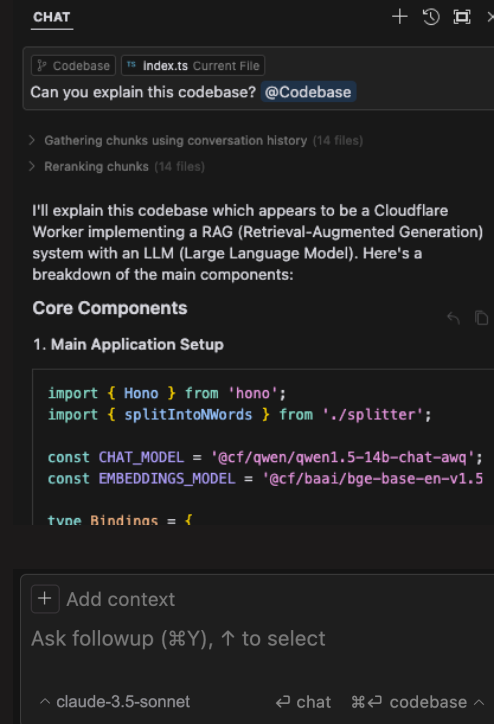
Lets you ask questions about your codebase and get context-aware responses

## 1. Chat Interface

- Ask questions about your code
- Debug issues conversationally
- Get implementation suggestions
- Discuss architectural decisions

## 2. Examples using Natural Language Commands

- Edit code using English descriptions
- Generate code from comments
- Refactor with text instructions
- Create documentation automatically
- Convert requirements to code



# Generate and Complete Code

Lets you generate code or complete code as you type

## 1. Generating Code

- Real-time suggestions as you type
- Multi-language support
- Context-aware completions
- Learning from your coding style

## 2. Examples of Code Generation

- Recommend variable names
- Complete or generate functions or methods
- Generate unit tests
- Implement design patterns
- Convert between languages
- Create boilerplate code
- Scaffold new components

# Code Updates

Lets you implement code suggestions from the chat directly into your codebase

- "Apply" button to implement suggestions
- "Apply All" to implement all suggestions
- Code is updated in the editor as diffs
  - You can review the changes before applying them
  - You can reject changes you don't like

```
{ } package.json  ⏪ Ask  📄 Copy  ▶ Apply

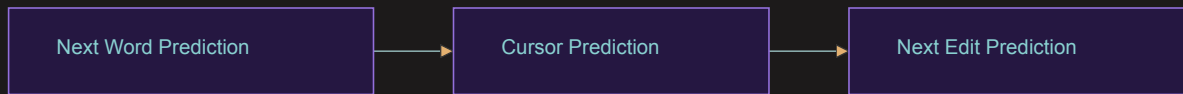
{
  "scripts": {
    "build-widget": "vite build"
  }
}
```

```
{ } package.json > ...
4   "scripts": {
12     "format": "prettier --write .",
13     "lint": "prettier --check . && eslint .",
14     "test:unit": "vitest",
15     "test": "npm run test:unit -- --run"
16     "test": "npm run test:unit -- --run",
17     "build-widget": "vite build"
18   },
19   "files": [
20     "dist",
21     "!dist/**/*.*.test.*"
  ]
```

# AI Prediction

AI Prediction is a feature that uses LLMs to predict what you will do next

- it started with Next Word Prediction
- then Cursor Prediction
- then Next Edit Prediction



- **Next Word Prediction:** Predicts the next word you will type.
- **Cursor Prediction:** Predicts where you will place your next cursor.
- **Next Edit Prediction:** Predicts the next edit you will make (refactor, add a function, etc).



# AI-Powered Code Review

Lets you review code and get suggestions for improvements

## 1. AI Assistants can review your code for you

- They generate suggestions and improvements
- They can even fix your code
- Tools: CodeRabbit, Greptile, Kluster.ai

## 2. GitHub Copilot code review

- Based on pull requests or local uncommitted changes
- Access from github.com or IDE

# Version Control and Collaboration

Lets you collaborate with others and manage your codebase

## 1. Real-time Collaboration

- Multi-user editing
- Shared AI context
- Team chat integration
- Code review assistance
- Knowledge sharing

## 2. Version Control Integration

- Git integration
- Commit message generation
- PR description writing
- Code review suggestions
- Merge conflict resolution

# Key Features: IDE Integration

## 1. Editor Features

- Syntax highlighting
- Auto-formatting
- Bracket matching
- Multi-cursor support
- Command palette
- Custom keybindings

## 2. Development Tools

- Terminal integration
- Debugger support
- Extension compatibility
- Theme customization
- Split views
- Project management

# Best Practices

## Effective Usage

### 1. Writing Clear Comments

- Be specific about desired functionality
- Include expected inputs/outputs
- Mention important constraints
- Reference relevant documentation

### 2. Context Management

- Keep relevant files open
- Position cursor appropriately
- Maintain clean project structure
- Use meaningful variable names

### 3. Review and Verify

- Always review generated code
- Test for edge cases
- Check for security issues
- Verify dependencies

# Common Pitfalls

## Watch Out For

### 1. Over-reliance

- Don't accept suggestions blindly
- Maintain understanding of the code
- Keep coding skills sharp
- Review security implications

### 2. Context Misunderstanding

- Incorrect framework versions
- Mixed programming paradigms
- Incompatible dependencies
- Wrong language features

### 3. Hallucinations

- Non-existent APIs
- Invalid syntax
- Fictional libraries
- Incorrect documentation

# Privacy and Security

## Key Considerations

### 1. Data Protection

- Code privacy settings
- Local processing options
- Data sharing controls
- Compliance requirements

### 2. Security Best Practices

- Review generated code
- Check for vulnerabilities
- Verify dependencies
- Monitor API usage

### 3. Team Settings

- Access controls
- Usage policies
- Data handling guidelines
- Collaboration protocols

# Future Developments

## Upcoming Trends

- Larger context windows
- Improved accuracy
- Better security features
- Enhanced collaboration tools
- More language support
- Custom model fine-tuning

## Integration Possibilities

- CI/CD pipeline integration
- Advanced testing capabilities
- Automated documentation
- Code review automation
- Performance optimization

# Getting Started

## 1. Choose Your Tool

- GitHub Copilot
- Cursor (which this course focuses on)
- Claude Code
- Others

## 2. Setup Process

- Install IDE extensions
- Configure settings
- Set up authentication
- Review privacy options

## 3. Initial Practice

- Start with simple tasks
- Experiment with features
- Learn keyboard shortcuts
- Build good habits early